

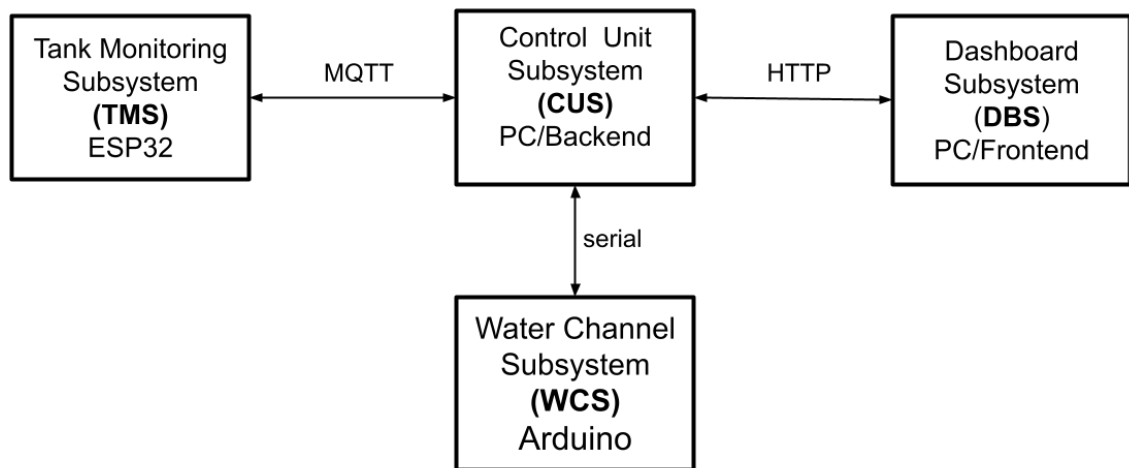
Report Assignment 03

Smart Tank Monitoring System

Studente: Pietro Siboni

1. Introduzione ed Architettura

Il progetto implementa un sistema IoT distribuito per il monitoraggio e il controllo di un canale d'acqua. L'architettura è suddivisa in 4 sottosistemi indipendenti che comunicano tra loro:



- **TMS:** Basato su ESP32-S3, monitora il livello dell'acqua nel serbatoio tramite sonar e comunica lo stato via Wi-Fi tramite protocollo MQTT.
- **CUS:** Applicazione in Java, si tratta del sottosistema principale che governa e coordina l'intero sistema. Riceve i dati MQTT dal TMS per il monitoraggio, elabora le Macchine a Stati e comunica via Seriale con il WCS. Funge da server HTTP per il DBS.
- **WCS:** Basato su Arduino Uno, sistema embedded che controlla il canale idrico che collega il serbatoio alla rete dei canali idrici. Interagisce tramite Seriale con il CUS. Gestisce l'attuatore fisico, un Servomotore che funge da valvola, mostra i dati su un LCD I2C e permette il controllo manuale tramite potenziometro e pulsante.
- **DBS:** Una Web App per operatori remoti. Interroga il CUS tramite HTTP e mostra un'interfaccia utente in tempo reale per visualizzare dati e interagire col sistema tramite l'invio di comandi manuali.

2. Schemi Hardware

2.1 Sottosistema TMS (ESP32-S3)

Il circuito del TMS include:

- un sensore **Sonar** per monitorare il livello dell'acqua;
- due **LED** di stato riguardanti la correttezza del sistema: Verde = connesso, Rosso = errore.

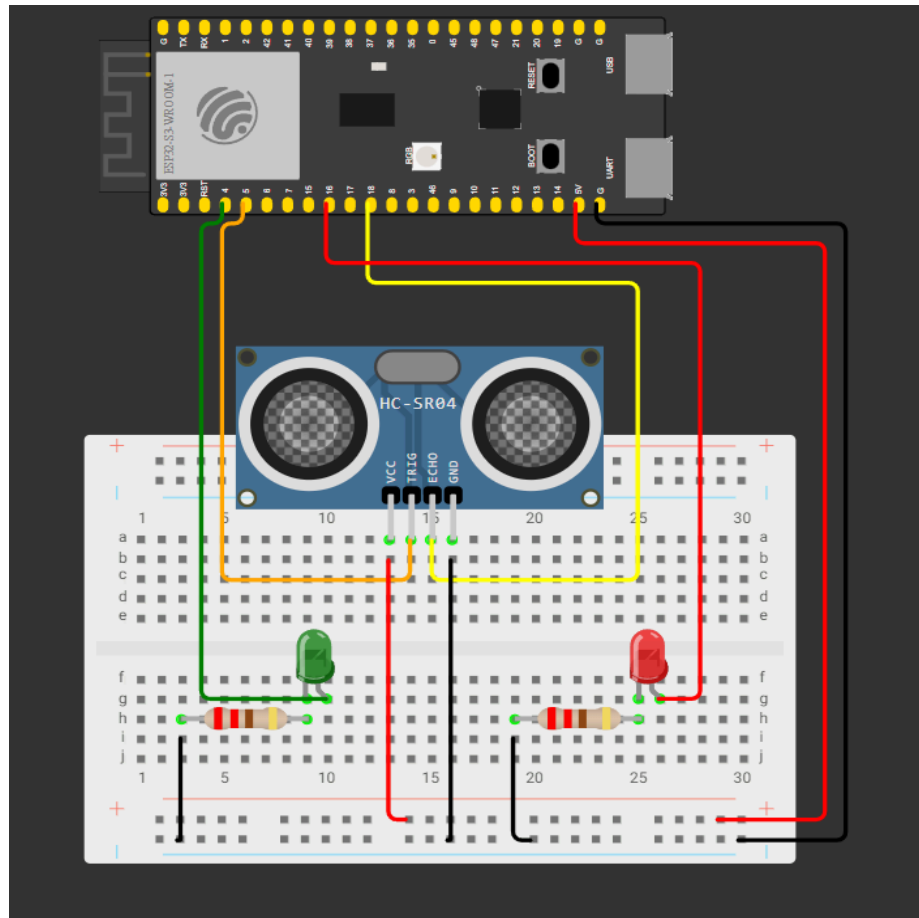


Figura 2.1: Schema breadboard circuito ESP32-S3.

2.2 Sottosistema WCS (Arduino Uno)

Il circuito del WCS include

- un **Servomotore** che simula il funzionamento di una valvola;
- un **Display LCD I2C** che mostra il livello di apertura attuale della valvola e la modalità (Automatic, Manual, Unconnected);
- un **Pulsante** per lo switch di modalità da Automatic a Manual;
- un **Potenzimetro** per l'apertura del manuale della valvola.

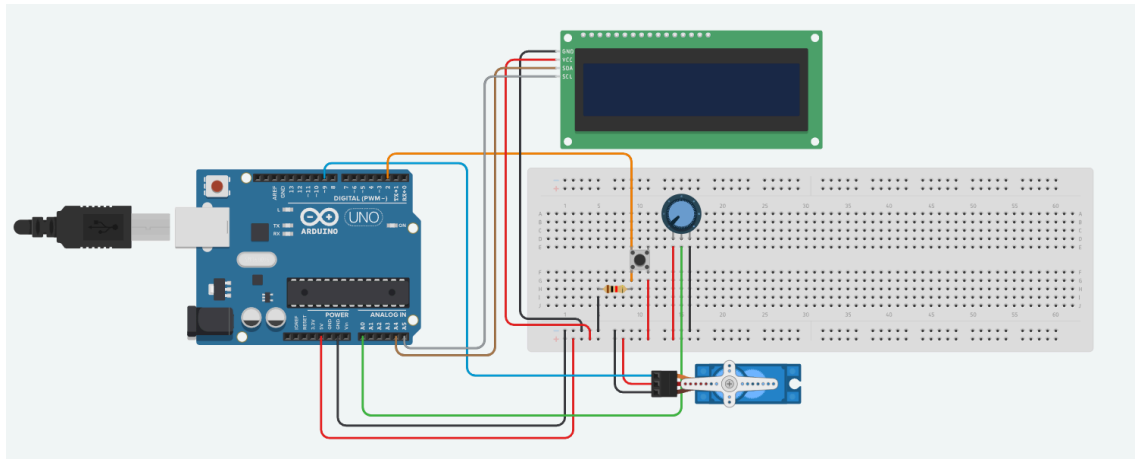


Figura 2.1: Schema breadboard circuito Arduino Uno.

3. Macchine a Stati Finiti (FSM)

La logica è stata suddivisa in più FSM comunicanti tra i quattro sottosistemi.

3.1 FSM CUS

Il CUS gestisce la logica di business principale attraverso due FSM concorrenti:

- FSM del **livello dell'acqua** quando si è in modalità automatica;
- FSM dello **stato di rete**.

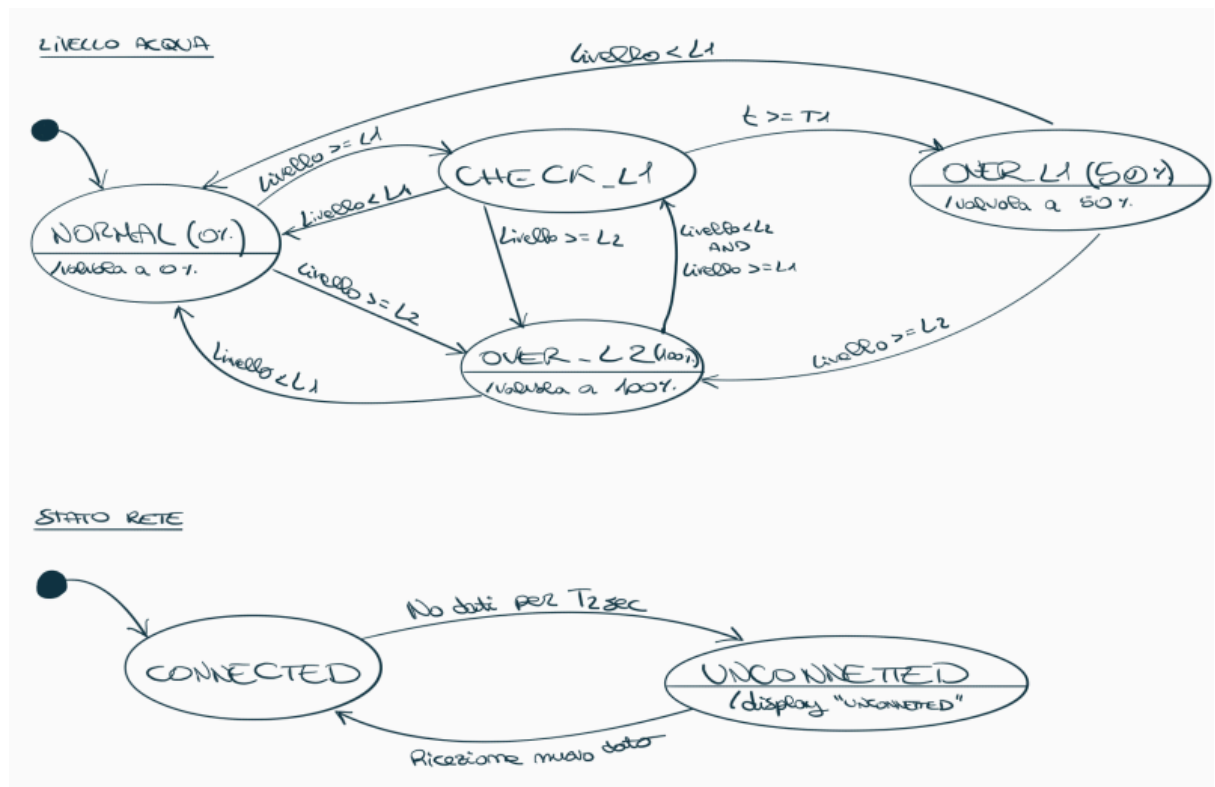


Figura 3.1: Rappresentazione del diagramma degli stati per le due FSM CUS.

3.2 FSM WCS

Il WCS gestisce l'interazione dell'utente attraverso la pressione di un pulsante per il cambio modalità tra Automatic e Manual.

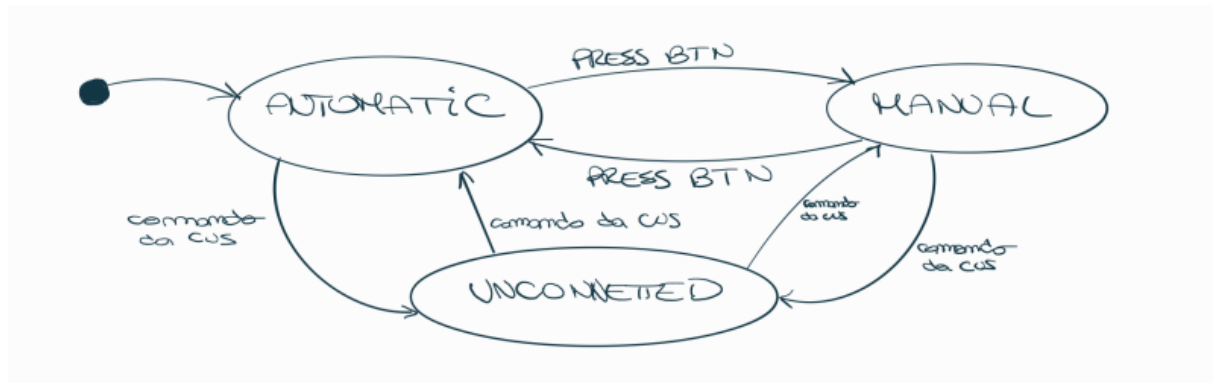


Figura 3.2: Rappresentazione del diagramma degli stati per la FSM WCS.

3.3 FSM TMS

Il TMS monitora la correttezza del sistema, ovvero se la rete e l'invio dati è funzionante determinandone i seguenti stati:

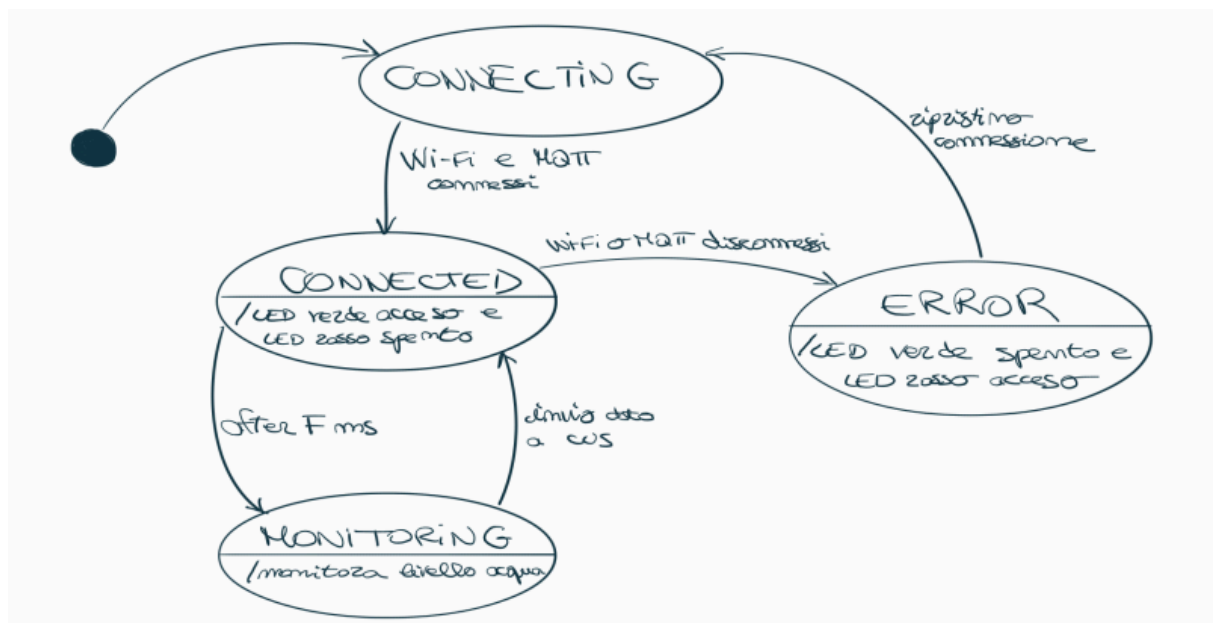


Figura 3.3: Rappresentazione del diagramma degli stati per la FSM TMS.

3.4 FSM DBS

Il DBS si occupa di verificare in primis la connessione HTTP tra sé e il CUS e nel caso di corretta connessione gestisce il cambio di modalità attraverso l'interfaccia utente.

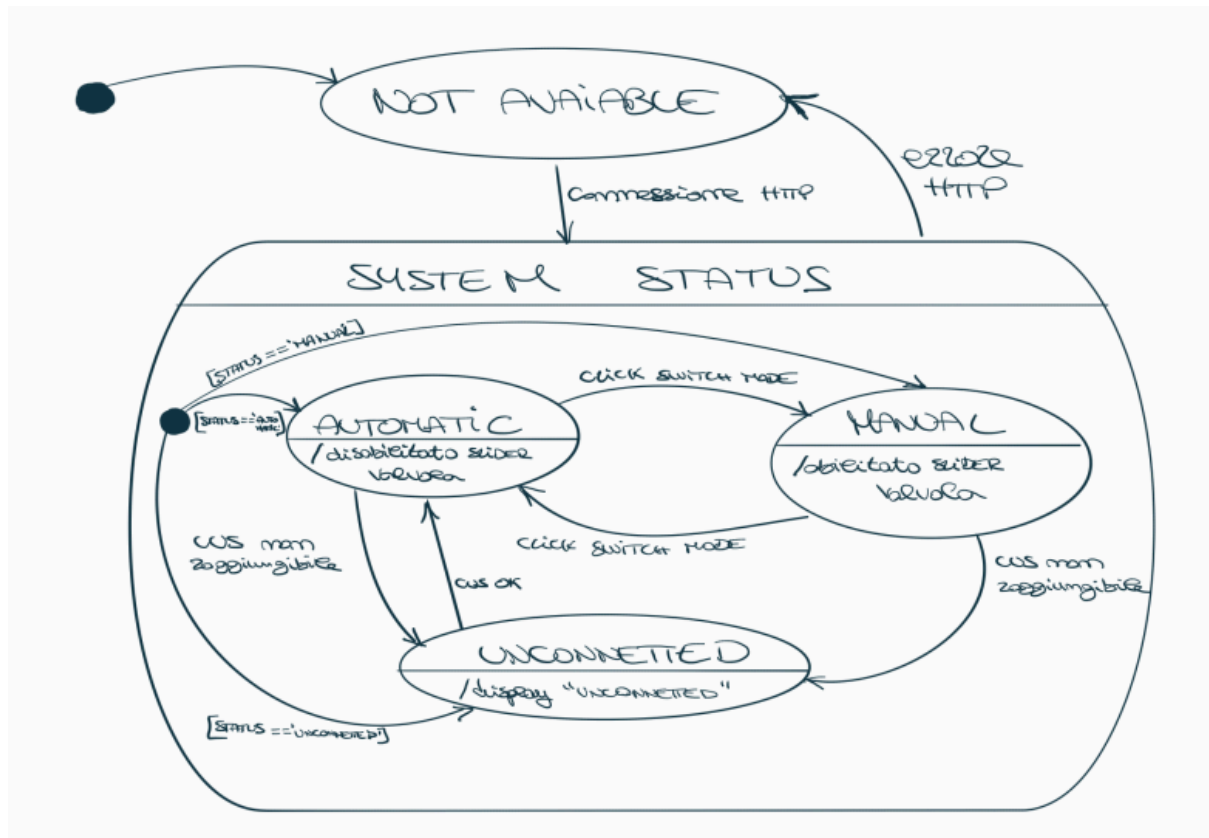


Figura 3.4: Rappresentazione del diagramma degli stati per la FSM DBS.

4. Dettagli Implementativi dei Sottosistemi

4.1 Tank Monitoring Subsystem (TMS)

Il modulo TMS, implementato su ESP32-S3, è stato programmato in C++.

Per gestire in parallelo la lettura del sensore e lo stato di connessione della rete è stato impiegato il sistema operativo FreeRTOS integrato tramite la funzione `xTaskCreatePinnedToCore` per integrarli al Core 1 del microcontrollore.

Il sottosistema è diviso in due Task principali:

- **networkTask:** Gestisce la connessione al Wi-Fi e si connette al broker MQTT pubblico (broker.mqtt-dashboard.com) tramite la libreria PubSubClient. Il task tenta ciclicamente la riconnessione in caso di caduta di rete aggiornando di conseguenza lo stato di connessione globale.
- **sensorTask:** Implementa la Macchina a Stati Finiti in [Figura 3.3](#); legge i dati dal Sonar e pilota i Led in base allo stato. L'invio del dato tramite MQTT ogni F millisecondi avviene solo se lo stato è Connected.

4.2 Control Unit Subsystem (CUS)

Il CUS, sviluppato in Java, funge da responsabile della politica di comportamento del sistema. Gestisce tre canali di comunicazione simultanei:

- **Comunicazione MQTT:** riceve dati dal TMS. Se smette di ricevere dati entro un timeout T2, forza il sistema allo stato di Unconnected finché non ne riceverà uno nuovo.
- **FSM livello d'acqua:** Viene implementato il primo diagramma in [Figura 3.1](#) relativo al monitoraggio del livello dell'acqua. Gestisce le transizioni tra i vari stati e invia immediatamente il comando tramite Seriale verso Arduino.
- **Seriale e Server HTTP:** Utilizza la seriale per inviare i gradi della valvola e le ricezioni di cambio stato. Contemporaneamente espone un'API HTTP locale che formatta l'intero stato del sistema in file JSON, per l'utilizzo della Dashboard, e accetta richieste POST per forzare le aperture manuali remote.

4.3 Water Channel Subsystem (WCS)

Il WCS è basato su Arduino Uno e gestisce la parte fisica del sottosistema.

- Resta in ascolto sulla Seriale per i comandi in arrivo dal CUS. Gestisce lo stato operativo, come descritto in [Figura 3.2](#), tramite la pressione di un pulsante e in qualsiasi momento, se riceve un allarme di disconnessione, passa allo stato di Unconnected.
- Viene controllata l'apertura del Servomotore (valvola) mappando le percentuali in gradi. Durante lo stato Manual, l'input di apertura della valvola viene letto direttamente dal potenziometro.

- Presenta un display LCD I2C che viene sovrascritto per mostrare i dati essenziali: il livello di apertura attuale delle valvola e la modalità (Automatic, Manual o Unconnected).

4.4 Dashboard Subsystem (DBS)

La Dashboard è una singola pagina realizzata in HTML, CSS e JavaScript che reagisce dinamicamente agli stati del sistema.

- Effettua richieste asincrone ciclicamente al server HTTP del CUS per scaricare il JSON aggiornato ed in base ai valori ricevuti aggiorna la pagina.
- L'interfaccia rispetta la FSM in [Figura 3.4](#). L'utente può cambiare i bottoni per gli stati operativi Automatic e Manual, inviando di conseguenza un comando POST al backend per sbloccare o bloccare l'interfaccia per comandare a mano l'apertura della valvola.

Nel caso si sia in uno stato di Unconnected o Not Available, la Dashboard disabiliterà il pannello riguardante il Centro di controllo prevenendo l'invio di comandi verso il backend.